```
Output:
Array
(
    [0] => Alice
    [1] => Bob
    [2] => Rita
    [3] => Sue
    [4] => Russell
)
```

The splat operator `...` transforms the elements of `$myArr2` into individual values and supplies them to `array_push`, as if it had been called with four arguments (the target array and the three string elements of the second array). We must do this otherwise, the result will be that `$myArr2` is inserted as a sub-array of `$myArr1`.

PHP 7.4 added support for using the splat operator within an array expression. For example, `$myArr = ["Alice", "Bob", ...$myOtherArr];` would create an array with `"Alice"` and `"Bob"` as the first two elements, then every element of `$myOtherArr` following them (rather than `$myOtherArr` as a sub-array).

With `array_merge()`

Another useful function for combining arrays is `array_merge()`. For indexed arrays, this will return a new array with the elements of both arrays concatenated in the order in which they were combined, leaving the original arrays unmodified. The output of the following is identical to the previous example.

```
$myArr1 = ["Alice", "Bob"];
$myArr2 = ["Rita", "Sue", "Russell"];

$merge  = array_merge($myArr1, $myArr2);

print_r($merge);

Output:
Array
(
    [0] => Alice
    [1] => Bob
```